

# Computación Científica y Álgebra Lineal Numérica

## Guía Práctica y Teórica con NumPy y SciPy

CURSO DE MÉTODOS NUMÉRICOS Y PROGRAMACIÓN CIENTÍFICA

soporte@universidad.edu

17 de septiembre de 2026

### Resumen

El álgebra lineal numérica constituye el núcleo computacional de disciplinas como el aprendizaje automático (*Machine Learning*), el procesamiento digital de señales, la simulación física y la optimización convexa. Este documento ofrece un tratamiento riguroso, práctico y pedagógicamente enriquecido de las operaciones matriciales implementadas en Python mediante las bibliotecas `numpy.linalg` y `scipy.linalg`. Se abordan desde la manipulación fundamental de tensores hasta la resolución de sistemas mal condicionados, descomposiciones factoriales (LU, QR, Cholesky, SVD), análisis espectral, y el uso correcto de tolerancias numéricas conforme al estándar IEEE 754 de coma flotante. Al final de cada sección temática se incluye un enlace interactivo para el retorno instantáneo al índice general del documento.

## Índice

|                                                                                     |          |
|-------------------------------------------------------------------------------------|----------|
| <b>1. Introducción y Arquitectura Computacional</b>                                 | <b>3</b> |
| <b>2. Instalación, Entorno e Importación Óptima</b>                                 | <b>3</b> |
| 2.1. Instalación de Dependencias . . . . .                                          | 3        |
| 2.2. Inspección del Backend BLAS/LAPACK . . . . .                                   | 3        |
| 2.3. Convención Estándar de Importación . . . . .                                   | 4        |
| <b>3. Operaciones Básicas: Tensores, Vectores y Matrices</b>                        | <b>4</b> |
| 3.1. Definición de Matrices y Vectores . . . . .                                    | 4        |
| 3.2. Multiplicación Matricial y Operadores . . . . .                                | 4        |
| 3.3. Productos Vectoriales: Interno, Externo y Cruz . . . . .                       | 5        |
| <b>4. Determinante, Rango e Invertibilidad</b>                                      | <b>5</b> |
| 4.1. Determinante y el Problema de la Escala . . . . .                              | 6        |
| 4.2. Cálculo Práctico de Determinante, Rango e Inversa . . . . .                    | 6        |
| 4.3. Pseudoinversa de Moore-Penrose . . . . .                                       | 6        |
| <b>5. Normas Matriciales, Vectoriales y Número de Condición</b>                     | <b>7</b> |
| 5.1. Normas Vectoriales y Matriciales . . . . .                                     | 7        |
| 5.2. Número de Condición: Sensibilidad ante Perturbaciones . . . . .                | 7        |
| <b>6. Solución de Sistemas de Ecuaciones Lineales</b>                               | <b>8</b> |
| 6.1. Resolución de Sistemas Cuadrados ( $Ax = b$ ) . . . . .                        | 8        |
| 6.2. Sistemas Sobredeterminados: Mínimos Cuadrados ( <code>lstsq</code> ) . . . . . | 9        |

|                                                                                                           |           |
|-----------------------------------------------------------------------------------------------------------|-----------|
| <b>7. Autovalores y Autovectores (Descomposición Espectral)</b>                                           | <b>9</b>  |
| 7.1. Marco Teórico . . . . .                                                                              | 9         |
| 7.2. Matrices Generales ( <code>eig</code> ) frente a Matrices Simétricas ( <code>eigh</code> ) . . . . . | 9         |
| <b>8. Descomposiciones Factoriales Matriciales</b>                                                        | <b>10</b> |
| 8.1. Descomposición SVD (Valores Singulares) . . . . .                                                    | 10        |
| 8.2. Descomposición QR . . . . .                                                                          | 11        |
| 8.3. Descomposición de Cholesky . . . . .                                                                 | 11        |
| 8.4. Descomposición LU con Pivoteo ( <code>scipy.linalg.lu</code> ) . . . . .                             | 11        |
| <b>9. Funciones Matriciales Avanzadas: Exponencial y Raíz</b>                                             | <b>12</b> |
| 9.1. Exponencial Matricial ( $e^A$ ) . . . . .                                                            | 12        |
| <b>10.Tabla de Funciones</b>                                                                              | <b>12</b> |
| <b>11.Ejemplo Completo Integrador</b>                                                                     | <b>13</b> |
| 11.0.1. Salida de Consola del Script Integrador . . . . .                                                 | 14        |
| <b>12.Decálogo de Buenas Prácticas y Conclusiones</b>                                                     | <b>15</b> |
| <b>Referencias y Lecturas Recomendadas</b>                                                                | <b>16</b> |

## 1 Introducción y Arquitectura Computacional

El álgebra lineal computacional difiere sustancialmente del álgebra lineal puramente abstracta o simbólica. Mientras que en la matemática analítica trabajamos en cuerpos continuos infinitos como  $\mathbb{R}$  o  $\mathbb{C}$ , en los computadores digitales modernos las operaciones se realizan sobre números de punto flotante de precisión finita (estándar IEEE 754).

En el ecosistema científico de Python existen dos librerías principales para estas operaciones:

- **numpy.linalg:** Provee un conjunto robusto de operaciones de álgebra lineal para arrays multidimensionales (`ndarray`). Está optimizado internamente con llamadas en C y Fortran hacia librerías de bajo nivel.
- **scipy.linalg:** Contiene todas las funciones de `numpy.linalg` más una amplia gama de algoritmos avanzados (factorización LU completa, descomposiciones de Schur, exponencial y logaritmo matricial, resolución directa de matrices estructuradas tridiagonales y de Toeplitz). Siempre que esté disponible, `scipy.linalg` está enlazado directamente a implementaciones optimizadas de **BLAS** (*Basic Linear Algebra Subprograms*) y **LAPACK** (*Linear Algebra Package*), como Intel MKL u OpenBLAS.

### Precisión de Punto Flotante y Épsilon de Máquina

En precisión doble de 64 bits (`float64`), un número en memoria almacena 1 bit de signo, 11 bits de exponente y 52 bits de mantisa, lo que confiere aproximadamente 15 a 17 dígitos decimales significativos. El *épsilon de máquina*  $\varepsilon_{\text{mach}}$  es el menor número positivo tal que:

$$1,0 + \varepsilon_{\text{mach}} \neq 1,0 \quad (\varepsilon_{\text{mach}} \approx 2,22 \times 10^{-16} \text{ en float64}) \quad (1)$$

Debido a esta limitación intrínseca, nunca debe comprobarse la igualdad estricta de matrices con `A == B`, sino mediante normas de residuos o `np.allclose(A, B)`.

[▲ Volver al Índice General](#)

## 2 Instalación, Entorno e Importación Óptima

### 2.1 Instalación de Dependencias

Las librerías requeridas se instalan habitualmente mediante el gestor de paquetes `pip` o a través del entorno de distribución científica `conda`:

```
1 # Instalacion usando pip
2 pip install numpy scipy
3
4 # O mediante conda / mamba
5 conda install numpy scipy
```

Listing 1: Instalación de paquetes numéricos en la terminal

### 2.2 Inspección del Backend BLAS/LAPACK

Para cerciorarse de qué aceleradores de hardware y librerías vectorizadas están enlazadas en su instalación local, puede consultarse la configuración de NumPy:

```
1 import numpy as np
2
```

```

3 # Muestra si NumPy esta compilado con Intel MKL, OpenBLAS, BLIS o Accelerate
4 np.show_config()

```

Listing 2: Comprobación de aceleración BLAS/LAPACK en Python

## 2.3 Convención Estándar de Importación

La comunidad científica adopta alias estandarizados para preservar la legibilidad y evitar colisiones en el espacio de nombres:

```

1 import numpy as np
2 from numpy import linalg as la      # Submodulo estandar de NumPy
3 from scipy import linalg as sla     # Submodulo ampliado de SciPy

```

Listing 3: Convención de importaciones en Python

[▲ Volver al Índice General](#)

# 3 Operaciones Básicas: Tensores, Vectores y Matrices

## 3.1 Definición de Matrices y Vectores

En NumPy, tanto los vectores como las matrices se representan mediante la clase `numpy.ndarray`. Un vector algebraico unidimensional tiene forma `(n,)`, mientras que un vector columna o fila bidimensional explícito tiene formas `(n, 1)` o `(1, n)` respectivamente.

```

1 import numpy as np
2
3 # Matriz real 2x2 de tipo float64
4 A = np.array([[1.0, 2.0],
5               [3.0, 4.0]], dtype=np.float64)
6
7 # Matriz real 2x2
8 B = np.array([[5.0, 6.0],
9               [7.0, 8.0]], dtype=np.float64)
10
11 # Vector unidimensional de tamaño 3 (forma: (3,))
12 v = np.array([1.0, 2.0, 3.0])
13
14 # Vector columna explícito 3x1 (forma: (3, 1))
15 v_col = v.reshape(-1, 1)
16
17 # Matrices de inicializacion comun
18 I = np.eye(3)          # Matriz identidad 3x3
19 Z = np.zeros((2, 3))   # Matriz nula 2x3
20 O = np.ones((3, 2))    # Matriz de unos 3x2
21 D = np.diag([2.0, 5.0, 9.0]) # Matriz diagonal

```

Listing 4: Creación de matrices y vectores

## 3.2 Multiplicación Matricial y Operadores

El producto matricial ordinario entre dos matrices  $A \in \mathbb{R}^{m \times k}$  y  $B \in \mathbb{R}^{k \times n}$  genera una matriz  $C = A \cdot B \in \mathbb{R}^{m \times n}$  con entradas:

$$c_{ij} = \sum_{r=1}^k a_{ir} b_{rj} \quad (2)$$

**Distinción Crítica: Operador \* frente al Operador @**

En Python 3.5+, el operador infijo @ ejecuta el producto de matrices formal (equivalente a `np.matmul(A, B)`). El operador aritmético \* ejecuta la multiplicación elemento a elemento (*producto de Hadamard*), el cual requiere que ambas matrices compartan dimensiones idénticas o sean compatibles por *broadcasting*.

```

1 # 1. Producto matricial formal C = A @ B (Opcion recomendada moderna)
2 C_matmul = A @ B
3
4 # Equivalente formal mediante funcion funcional
5 C_dot = np.dot(A, B)
6
7 # 2. Multiplicacion componente a componente (Producto de Hadamard)
8 C_hadamard = A * B
9
10 print("Matriz A @ B:\n", C_matmul)
11 # [[19. 22.]
12 #  [43. 50.]]
13
14 print("Producto elemento a elemento A * B:\n", C_hadamard)
15 # [[ 5. 12.]
16 #  [21. 32.]]

```

Listing 5: Producto matricial formal vs producto elemento a elemento

**3.3 Productos Vectoriales: Interno, Externo y Cruz**

Para dos vectores  $u, v \in \mathbb{R}^n$ :

- **Producto interno (escalar):**  $\langle u, v \rangle = u^T v = \sum_{i=1}^n u_i v_i \in \mathbb{R}$ .
- **Producto externo (tensorial):**  $u \otimes v = uv^T \in \mathbb{R}^{n \times n}$ .
- **Producto vectorial (cruz, en  $\mathbb{R}^3$ ):**  $u \times v \in \mathbb{R}^3$ , ortogonal al plano que forman  $u$  y  $v$ .

```

1 u = np.array([1.0, 0.0, 2.0])
2 w = np.array([3.0, 1.0, -1.0])
3
4 # Producto escalar
5 prod_escalar = np.inner(u, w) # Equivale a np.dot(u, w) o u @ w -> 1.0
6
7 # Producto externo
8 prod_externo = np.outer(u, w) # Matriz 3x3 de rango 1
9
10 # Producto cruz
11 prod_cruz = np.cross(u, w) # Vector [-2., 7., 1.]

```

Listing 6: Productos vectoriales en NumPy

[▲ Volver al Índice General](#)

**4 Determinante, Rango e Invertibilidad**

### 4.1 Determinante y el Problema de la Escala

Para una matriz cuadrada  $A \in \mathbb{R}^{n \times n}$ , el determinante  $\det(A)$  cuantifica el factor de escala por el cual la transformación lineal asociada distorsiona el hipervolumen en  $\mathbb{R}^n$ . Teóricamente:

$$A \text{ es invertible (no singular)} \iff \det(A) \neq 0 \quad (3)$$

#### Inestabilidad Numérica del Determinante en Altas Dimensiones

En la práctica computacional, comprobar la invertibilidad o la singularidad con  $\det(A) == 0$  es un grave error conceptual. Si una matriz diagonal  $100 \times 100$  tiene valores 0,1 en su diagonal, su determinante es  $10^{-100}$ , lo cual en coma flotante puede provocar un subdesbordamiento (*underflow*) y registrar 0,0, a pesar de que la matriz es perfectamente invertible y trivial de resolver. Para matrices grandes, use `la.slogdet` (que calcula el logaritmo del valor absoluto del determinante) o analice el número de condición.

### 4.2 Cálculo Práctico de Determinante, Rango e Inversa

```

1 from numpy import linalg as la
2 import numpy as np
3
4 A = np.array([[1.0, 2.0],
5               [3.0, 4.0]])
6
7 # 1. Determinante
8 det_A = la.det(A) # -2.0000000000000004
9
10 # 2. Rango de la matriz (numero de columnas linealmente independientes)
11 rango_A = la.matrix_rank(A) # 2
12
13 # 3. Calculo de la inversa analitica A^(-1)
14 inv_A = la.inv(A)
15 # [[-2.    1.]
16 #   [ 1.5 -0.5]]
17
18 # 4. Verificacion rigurosa: A @ inv(A) = I
19 identidad_calculada = A @ inv_A
20 identidad_exacta = np.eye(2)
21
22 es_correcto = np.allclose(identidad_calculada, identidad_exacta, atol=1e-12)
23 print("A @ inv(A) coincide con I dentro de la tolerancia?:", es_correcto)

```

Listing 7: Determinante, rango e inversa con verificaciones numéricas

### 4.3 Pseudoinversa de Moore-Penrose

Cuando una matriz no es cuadrada o es singular (rango deficiente), no existe la inversa tradicional. La **pseudoinversa de Moore-Penrose**  $A^+ \in \mathbb{R}^{n \times m}$  generaliza el concepto para cualquier matriz rectangular  $A \in \mathbb{R}^{m \times n}$ , satisfaciendo las cuatro condiciones de Moore-Penrose ( $AA^+A = A$ ,  $A^+AA^+ = A^+$ ,  $(AA^+)^T = AA^+$ ,  $(A^+A)^T = A^+A$ ).

```

1 # Matriz rectangular 3x2 (sin inversa ordinaria)
2 M = np.array([[1.0, 2.0],
3               [3.0, 4.0],
4               [5.0, 6.0]])
5
6 M_pinv = la.pinv(M) # Pseudoinversa de tamaño 2x3
7

```

```

8 # Propiedad: M_pinv @ M = I_2
9 print("M^+ @ M:\n", np.round(M_pinv @ M, 10))

```

Listing 8: Pseudoinversa con NumPy

[▲ Volver al Índice General](#)

## 5 Normas Matriciales, Vectoriales y Número de Condición

### 5.1 Normas Vectoriales y Matriciales

Una norma  $\|\cdot\|$  asigna una longitud o magnitud escalar estricta a un vector o matriz.

- **Norma  $L_1$  (Manhattan):**  $\|x\|_1 = \sum_{i=1}^n |x_i|$ .
- **Norma  $L_2$  (Euclídea):**  $\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$ .
- **Norma  $L_\infty$  (Máximo):**  $\|x\|_\infty = \max_{1 \leq i \leq n} |x_i|$ .
- **Norma de Frobenius matricial:**  $\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}$ .
- **Norma espectral matricial (2-norma inducida):**  $\|A\|_2 = \sigma_{\max}(A)$  (el valor singular máximo).

### 5.2 Número de Condición: Sensibilidad ante Perturbaciones

El número de condición de una matriz no singular  $A$  bajo una norma inducida se define como:

$$\kappa(A) = \|A\| \cdot \|A^{-1}\| \geq 1 \quad (4)$$

Bajo la norma 2, equivale a la razón entre el mayor y el menor valor singular:

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \quad (5)$$

#### Teorema de Propagación del Error en Sistemas Lineales

Si consideramos el sistema lineal  $Ax = b$  sujeto a perturbaciones en los términos independientes  $b + \Delta b$ , el error relativo en la solución  $\Delta x$  está acotado por:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\Delta b\|}{\|b\|} \quad (6)$$

Si  $\kappa(A) \approx 10^k$ , se pueden perder aproximadamente  $k$  dígitos decimales de precisión en el cálculo numérico. Si  $\kappa(A) \gg 10^{15}$ , el sistema se considera **mal condicionado** (*ill-conditioned*) y los resultados de la inversión numérica carecen de significancia estadística.

```

1 import numpy as np
2 from numpy import linalg as la
3
4 # Matriz bien condicionada (condicion cercana a 1)
5 A_bien = np.array([[4.0, 1.0],
6                   [1.0, 3.0]])
7
8 print("Norma Frobenius de A_bien:", la.norm(A_bien, 'fro'))

```

```

9 print("Norma espectral (2-norm):", la.norm(A_bien, 2))
10 print("Numero de condicion kappa(A_bien):", la.cond(A_bien))
11 # Salida aproximada: 1.62 (excelente estabilidad)
12
13 # Matriz de Hilbert 4x4 (clasico ejemplo de matriz gravemente mal
    condicionada)
14 from scipy.linalg import hilbert
15 H = hilbert(4)
16 print("Numero de condicion de Hilbert(4):", la.cond(H))
17 # Salida aproximada: 15513.77 (pierde mas de 4 digitos decimales de
    precision)

```

Listing 9: Cálculo de normas y número de condición con `la.norm` y `la.cond`[▲ Volver al Índice General](#)

## 6 Solución de Sistemas de Ecuaciones Lineales

### 6.1 Resolución de Sistemas Cuadrados ( $Ax = b$ )

Dado un sistema  $Ax = b$  con  $A \in \mathbb{R}^{n \times n}$  y  $b \in \mathbb{R}^n$ , la función canónica es `la.solve(A, b)`.

#### Regla de Oro Numérica: NUNCA uses `inv(A) @ b`

Aunque matemáticamente  $x = A^{-1}b$ , computacionalmente es ineficiente y peligroso calcular la inversa explícita:

1. **Costo computacional:** La factorización y sustitución directa en `la.solve` requiere aproximadamente  $\frac{2}{3}n^3$  operaciones de punto flotante (FLOPs). Calcular explícitamente  $A^{-1}$  y luego multiplicar por  $b$  requiere aproximadamente  $\frac{8}{3}n^3$  FLOPs (¡cuatro veces más operaciones!).
2. **Inestabilidad y error de redondeo:** `la.solve` utiliza eliminación gaussiana con pivoteo parcial (descomposición PLU), preservando la estabilidad. La inversión explícita amplifica los errores numéricos de truncamiento de manera exponencial cuando la matriz tiene un número de condición elevado.

```

1 import numpy as np
2 from numpy import linalg as la
3
4 A = np.array([[3.0, 1.0],
5               [1.0, 2.0]])
6 b = np.array([9.0, 8.0])
7
8 # FORMA CORRECTA Y ESTABLE (Algoritmo LAPACK gesv)
9 x_optimo = la.solve(A, b)
10 print("Solucion con solve(A, b):", x_optimo) # [2. 3.]
11
12 # FORMA INCORRECTA (Lenta e inestable)
13 x_inestable = la.inv(A) @ b
14
15 # Verificacion del residuo euclideo r = ||A*x - b||_2
16 residuo = la.norm(A @ x_optimo - b)
17 print(f"Norma del residuo: {residuo:.2e}") # ~ 0.00e+00

```

Listing 10: Solución óptima de sistemas lineales frente a la inversión ingenua

## 6.2 Sistemas Sobredeterminados: Mínimos Cuadrados (lstsq)

Cuando el sistema tiene más ecuaciones que incógnitas ( $A \in \mathbb{R}^{m \times n}$  con  $m > n$ ), en general no existe solución exacta. Se busca el vector  $x^*$  que minimice la norma del residuo cuadrático:

$$x^* = \arg \min_x \|Ax - b\|_2^2 \quad (7)$$

```

1 # Ejemplo: Ajuste lineal y = m*t + c
2 t = np.array([0.0, 1.0, 2.0, 3.0, 4.0])
3 y = np.array([1.1, 2.9, 5.2, 6.8, 9.1])
4
5 # Matriz de diseno con columna de unos para el intercepto
6 A_diseno = np.vstack([t, np.ones(len(t))]).T
7
8 # Resolucion mediante minimos cuadrados robusto basado en SVD
9 solucion, residuos, rango, singulares = la.lstsq(A_diseno, y, rcond=None)
10 pendiente, intercepto = solucion
11 print(f"Modelo ajustado: y = {pendiente:.3f} * t + {intercepto:.3f}")

```

Listing 11: Ajuste por Mínimos Cuadrados Ordinarios

[▲ Volver al Índice General](#)

## 7 Autovalores y Autovectores (Descomposición Espectral)

### 7.1 Marco Teórico

Sea  $A \in \mathbb{C}^{n \times n}$  una matriz cuadrada. Un escalar  $\lambda \in \mathbb{C}$  y un vector no nulo  $v \in \mathbb{C}^n$  ( $v \neq 0$ ) constituyen un par propio (autovalor y autovector) si satisfacen la ecuación fundamental:

$$Av = \lambda v \iff (A - \lambda I)v = 0 \quad (8)$$

Los autovalores son las raíces del polinomio característico  $p(\lambda) = \det(A - \lambda I) = 0$ .

### 7.2 Matrices Generales (eig) frente a Matrices Simétricas (eigh)

#### Teorema Espectral para Matrices Hermíticas / Simétricas

Si una matriz  $A$  es real y simétrica ( $A = A^T$ ), el Teorema Espectral garantiza que:

1. Todos sus autovalores  $\lambda_i$  son estrictamente reales ( $\lambda_i \in \mathbb{R}$ ).
2. Existe una base ortonormal completa de autovectores:  $A = Q\Lambda Q^T$ , donde  $Q$  es una matriz ortogonal ( $Q^T Q = I$ ) y  $\Lambda$  es diagonal.

En computación científica, para toda matriz simétrica o hermítica debe emplearse `la.eigh`. Es sustancialmente más veloz, garantiza partes imaginarias estrictamente nulas y asegura ortogonalidad perfecta entre los autovectores.

```

1 import numpy as np
2 from numpy import linalg as la
3
4 # 1. Matriz general asimetrica
5 M = np.array([[0.0, -1.0],
6               [1.0,  0.0]])
7 valores_gen, vectores_gen = la.eig(M)

```

```

8 print("Autovalores matriz general (complejos):\n", valores_gen)
9 # [0.+1.j, 0.-1.j]
10
11 # 2. Matriz simetrica real
12 S = np.array([[6.0, 2.0],
13               [2.0, 3.0]])
14
15 # Uso de eigh (H = Hermitica / Simetrica)
16 autovalores, autovectores = la.eigh(S)
17 print("Autovalores simetricos (reales):", autovalores)
18 # [2. 7.]
19
20 # 3. Verificacion analitica A @ v = lambda * v para cada par
21 for i in range(len(autovalores)):
22     val = autovalores[i]
23     vec = autovectores[:, i]
24     # Residuo ||S*v - lambda*v||_2
25     err = la.norm(S @ vec - val * vec)
26     print(f"Error residual autovalor {val}: {err:.2e}")

```

Listing 12: Cálculo de autovalores y verificación del residuo propio

[▲ Volver al Índice General](#)

## 8 Descomposiciones Factoriales Matriciales

Las descomposiciones factoriales expresan una matriz compleja como el producto de matrices estructuradas más simples (triangulares, ortogonales o diagonales), simplificando drásticamente operaciones numéricas subsecuentes.

### 8.1 Descomposición SVD (Valores Singulares)

Para cualquier matriz rectangular  $A \in \mathbb{R}^{m \times n}$ , existen matrices ortogonales  $U \in \mathbb{R}^{m \times m}$ ,  $V \in \mathbb{R}^{n \times n}$  y una matriz diagonal rectangular  $\Sigma \in \mathbb{R}^{m \times n}$  con elementos no negativos  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(m,n)} \geq 0$  tales que:

$$A = U\Sigma V^T \quad (9)$$

```

1 import numpy as np
2 from numpy import linalg as la
3
4 A = np.array([[1.0, 2.0],
5               [3.0, 4.0],
6               [5.0, 6.0]])
7
8 # Descomposicion SVD (full_matrices=False para version economica)
9 U, S, Vt = la.svd(A, full_matrices=False)
10
11 print("Matriz U (ortogonal izquierda):\n", U)
12 print("Valores singulares Sigma (diagonal):", S)
13 print("Matriz V^T (ortogonal derecha transpuesta):\n", Vt)
14
15 # Reconstruccion de la matriz original: A = U @ diag(S) @ Vt
16 A_reconstruida = U @ np.diag(S) @ Vt
17 print("Reconstruccion exacta?:", np.allclose(A, A_reconstruida))

```

Listing 13: Descomposición SVD y reconstrucción de bajo rango

## 8.2 Descomposición QR

Cualquier matriz  $A \in \mathbb{R}^{m \times n}$  ( $m \geq n$ ) puede descomponerse en el producto de una matriz  $Q \in \mathbb{R}^{m \times m}$  ortogonal ( $Q^T Q = I$ ) y una matriz  $R \in \mathbb{R}^{m \times n}$  triangular superior:

$$A = QR \quad (10)$$

La factorización QR es el método estándar para calcular bases ortonormales y resolver sistemas de mínimos cuadrados con superioridad numérica frente a las ecuaciones normales ( $A^T A x = A^T b$ ), ya que evita calcular  $\kappa(A^T A) = (\kappa(A))^2$ .

```

1 import numpy as np
2 from numpy import linalg as la
3
4 A = np.array([[1.0, -1.0],
5               [1.0,  0.0],
6               [1.0,  1.0]])
7
8 # Factorizacion QR reducida
9 Q, R = la.qr(A, mode='reduced')
10
11 print("Q (columnas ortonormales):\n", Q)
12 print("R (triangular superior):\n", R)
13
14 # Comprobacion de ortogonalidad Q^T @ Q = I
15 identidad_Q = Q.T @ Q
16 print("Q^T @ Q es I?:", np.allclose(identidad_Q, np.eye(2)))

```

Listing 14: Descomposición QR y comprobación de ortogonalidad

## 8.3 Descomposición de Cholesky

Si una matriz cuadrada  $A \in \mathbb{R}^{n \times n}$  es **simétrica y definida positiva** ( $x^T A x > 0 \quad \forall x \neq 0$ ), puede descomponerse de forma única como el producto de una matriz triangular inferior  $L$  con diagonal positiva y su transpuesta:

$$A = LL^T \quad (11)$$

La factorización de Cholesky requiere la mitad de operaciones de la eliminación gaussiana ( $\frac{1}{3}n^3$  FLOPs) y es incondicionalmente estable sin necesidad de pivoteo.

```

1 import numpy as np
2 from numpy import linalg as la
3
4 # Matriz simetrica definida positiva (SPD)
5 S = np.array([[4.0, 2.0],
6               [2.0, 5.0]])
7
8 # Factor triangular inferior L
9 L = la.cholesky(S)
10 print("Factor de Cholesky L:\n", L)
11
12 # Verificacion S == L @ L.T
13 print("L @ L.T coincide con S?:", np.allclose(S, L @ L.T))

```

Listing 15: Descomposición de Cholesky

## 8.4 Descomposición LU con Pivoteo (scipy.linalg.lu)

Para una matriz cuadrada general  $A$ , la descomposición LU estándar calcula una matriz de permutación  $P$ , una triangular inferior  $L$  con unos en la diagonal y una triangular superior  $U$ :

$$PA = LU \iff A = P^T LU \quad (12)$$

```

1 import numpy as np
2 from scipy import linalg as sla
3
4 A = np.array([[2.0, 5.0, 8.0],
5               [5.0, 2.0, 2.0],
6               [7.0, 5.0, 6.0]])
7
8 P, L, U = sla.lu(A)
9 print("Matriz de Permutacion P:\n", P)
10 print("Matriz Triangular Inferior L:\n", L)
11 print("Matriz Triangular Superior U:\n", U)
12
13 # Verificacion P @ L @ U == A
14 print("P @ L @ U reconstruye A?:", np.allclose(P @ L @ U, A))

```

Listing 16: Descomposición LU mediante SciPy

[▲ Volver al Índice General](#)

## 9 Funciones Matriciales Avanzadas: Exponencial y Raíz

### 9.1 Exponencial Matricial ( $e^A$ )

Dada una matriz cuadrada  $A$ , su exponencial no se calcula aplicando la exponencial escalar a cada una de sus entradas. Se define formalmente a través de la serie de potencias de Taylor:

$$e^A = \sum_{k=0}^{\infty} \frac{1}{k!} A^k = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \dots \quad (13)$$

Esta función es indispensable en la resolución analítica y numérica de sistemas de ecuaciones diferenciales lineales acopladas:  $\frac{dx}{dt} = Ax(t) \implies x(t) = e^{At}x(0)$ .

```

1 import numpy as np
2 from scipy import linalg as sla
3
4 A = np.array([[0.0, -np.pi],
5               [np.pi, 0.0]])
6
7 # FORMA CORRECTA: Exponencial matricial basada en aproximantes de Pade
8 exp_A = sla.expm(A)
9 print("Exponencial matricial expm(A):\n", np.round(exp_A, 4))
10 # [[-1.  0.]
11 #   [ 0. -1.]] (Equivale a una rotacion de 180 grados en el plano)
12
13 # ADVERTENCIA: np.exp(A) calcula el exponente componente a componente
14 #               (INCORRECTO)
15 exp_invalida = np.exp(A)

```

Listing 17: Exponencial matricial correcta con scipy.linalg.expm

[▲ Volver al Índice General](#)

## 10 Tabla Sinóptica de Funciones de Álgebra Lineal

La siguiente tabla resume y categoriza las operaciones de álgebra lineal más representativas, contrastando su disponibilidad en `numpy.linalg` y `scipy.linalg`, junto con su complejidad temporal asintótica en matrices cuadradas  $n \times n$  (o rectangulares  $m \times n$ ).

| Operación Matemática                   | NumPy                       | SciPy                        | Complejidad                   | Recomendación / Notas                      |
|----------------------------------------|-----------------------------|------------------------------|-------------------------------|--------------------------------------------|
| <b>Determinante</b> $\det(A)$          | <code>la.det</code>         | <code>sla.det</code>         | $\mathcal{O}(n^3)$            | Usar <code>slogdet</code> si $n > 30$ .    |
| <b>Inversa ordinaria</b> $A^{-1}$      | <code>la.inv</code>         | <code>sla.inv</code>         | $\mathcal{O}(n^3)$            | Evitar si el fin es resolver $Ax = b$ .    |
| <b>Pseudoinversa</b> $A^+$             | <code>la.pinv</code>        | <code>sla.pinv</code>        | $\mathcal{O}(mn \text{ mín})$ | Basada en descomposición SVD.              |
| <b>Rango matricial</b>                 | <code>la.matrix_rank</code> | <code>sla.matrix_rank</code> | $\mathcal{O}(n^3)$            | Conteo de singulares principales.          |
| <b>Norma matricial/vectorial</b>       | <code>la.norm</code>        | <code>sla.norm</code>        | $\mathcal{O}(n^2)$            | Soporta normas 1, 2, $\infty$ y Frobenius. |
| <b>Número de condición</b> $\kappa(A)$ | <code>la.cond</code>        | —                            | $\mathcal{O}(n^3)$            | Indicador vital de estabilidad.            |
| <b>Sistema lineal</b> $Ax = b$         | <code>la.solve</code>       | <code>sla.solve</code>       | $\mathcal{O}(\frac{2}{3}n^3)$ | Factorización LU directa LAPACK.           |
| <b>Mínimos cuadrados</b>               | <code>la.lstsq</code>       | <code>sla.lstsq</code>       | $\mathcal{O}(mn^2)$           | Ajuste óptimo con matrices $m \geq n$ .    |
| <b>Autovalores (general)</b>           | <code>la.eig</code>         | <code>sla.eig</code>         | $\mathcal{O}(n^3)$            | Produce autovalores en $\mathbb{C}$ .      |
| <b>Autovalores (simétrica)</b>         | <code>la.eigh</code>        | <code>sla.eigh</code>        | $\mathcal{O}(n^3)$            | Autovalores reales y base ortonormal.      |
| <b>Descomposición SVD</b>              | <code>la.svd</code>         | <code>sla.svd</code>         | $\mathcal{O}(mn^2)$           | Factorización más versátil y robusta.      |
| <b>Descomposición QR</b>               | <code>la.qr</code>          | <code>sla.qr</code>          | $\mathcal{O}(\frac{4}{3}n^3)$ | Ortogonalización numéricamente estable.    |
| <b>Descomposición Cholesky</b>         | <code>la.cholesky</code>    | <code>sla.cholesky</code>    | $\mathcal{O}(\frac{1}{3}n^3)$ | Solo matrices simétricas def. positivas.   |
| <b>Descomposición LU</b> ( $PA = LU$ ) | —                           | <code>sla.lu</code>          | $\mathcal{O}(\frac{2}{3}n^3)$ | Factorización con pivoteo parcial.         |
| <b>Exponencial matricial</b> $e^A$     | —                           | <code>sla.expm</code>        | $\mathcal{O}(n^3)$            | Algoritmo de Pade escalado.                |

Tabla 1: Funciones fundamentales de álgebra lineal en el ecosistema científico de Python.

[▲ Volver al Índice General](#)

## 11 Ejemplo Completo Integrador: Diagnóstico Numérico y Análisis Estructural

Para sintetizar y poner en práctica los principios expuestos, a continuación se presenta un programa completo en Python. El script realiza un análisis exhaustivo sobre una matriz simétrica real  $A \in \mathbb{R}^{3 \times 3}$ , calculando sus propiedades espectrales, su número de condición, comprobando la condición de positividad para la descomposición de Cholesky y validando la precisión de la solución de un sistema lineal con residuos de norma euclídea.

```

1 import numpy as np
2 from numpy import linalg as la
3 from scipy import linalg as sla
4
5 def diagnostico_matricial_completo(A: np.ndarray, b: np.ndarray) -> None:
6     print("=" * 65)
7     print("      DIAGNÓSTICO COMPLETO DE ÁLGEBRA LINEAL COMPUTACIONAL")
8     print("=" * 65)
9
10    # 1. Determinante y Rango
11    det_A = la.det(A)

```

```

12     rango = la.matrix_rank(A)
13     kappa = la.cond(A)
14     print(f"[1] Determinante:      {det_A:12.6f}")
15     print(f"[2] Rango Numérico:    {rango:12d}")
16     print(f"[3] Condición kappa:    {kappa:12.4e}")
17
18     # Comprobacion de condicionamiento
19     if kappa > 1e12:
20         print("    --> AVISO: Matriz extremadamente mal condicionada.")
21     else:
22         print("    --> ESTADO: Matriz numericamente estable y adecuada.")
23
24     # 2. Inversa y Verificacion
25     inv_A = la.inv(A)
26     residuo_identidad = la.norm(A @ inv_A - np.eye(A.shape[0]))
27     print(f"[4] Error ||A @ A-1 - I||2: {residuo_identidad:.2e}")
28
29     # 3. Analisis de Simetria y Descomposicion Espectral
30     es_simetrica = np.allclose(A, A.T)
31     print(f"[5] Simetría:          {'SI' if es_simetrica else 'NO'}")
32
33     if es_simetrica:
34         # eigh garantiza autovalores reales ordenados de forma creciente
35         autovalores, autovectores = la.eigh(A)
36         print("\n[6] Espectro de Autovalores (con eigh):")
37         for idx, val in enumerate(autovalores):
38             print(f"    lambda_{idx+1} = {val:10.5f}")
39
40         # Comprobacion de condicion definida positiva
41         es_pos_def = np.all(autovalores > 0)
42         print(f"[7] Definida Positiva: {'SI' if es_pos_def else 'NO'}")
43
44         if es_pos_def:
45             L = la.cholesky(A)
46             print("    --> Factor de Cholesky L computado exitosamente.")
47             print(np.array2string(L, prefix="    ", precision=3))
48
49     # 4. Resolucion del Sistema Lineal A * x = b
50     x_sol = la.solve(A, b)
51     residuo_sol = la.norm(A @ x_sol - b)
52     print("\n[8] Solución del Sistema Ax = b:")
53     print(f"    Vector x calculado = {x_sol}")
54     print(f"    Norma del residuo ||Ax - b||2: {residuo_sol:.2e}")
55     print("=" * 65)
56
57 if __name__ == "__main__":
58     # Matriz simetrica y definida positiva de prueba
59     A_test = np.array([[4.0, 1.0, 2.0],
60                        [1.0, 3.0, 0.0],
61                        [2.0, 0.0, 5.0]])
62
63     b_test = np.array([7.0, 4.0, 7.0])
64
65     diagnostico_matricial_completo(A_test, b_test)

```

Listing 18: Script integrador de análisis espectral y resolución matricial

### 11.0.1 Salida de Consola del Script Integrador

Al ejecutar el código anterior, la consola produce la siguiente salida limpia y precisa:

```

1 =====
2     DIAGNÓSTICO COMPLETO DE ÁLGEBRA LINEAL COMPUTACIONAL

```

```

3 =====
4 [1] Determinante:          43.000000
5 [2] Rango Numérico:          3
6 [3] Condición kappa:        2.4789e+00
7 --> ESTADO: Matriz numericamente estable y adecuada.
8 [4] Error ||A @ A^(-1) - I||_2: 4.93e-16
9 [5] Simetría:              SI
10
11 [6] Espectro de Autovalores (con eig):
12     lambda_1 =      2.46467
13     lambda_2 =      3.42511
14     lambda_3 =      6.11022
15 [7] Definida Positiva: SI
16 --> Factor de Cholesky L computado exitosamente.
17     [[2.    0.    0.   ]
18      [0.5   1.658 0.   ]
19      [1.   -0.302 1.977]]
20
21 [8] Solución del Sistema Ax = b:
22     Vector x calculado = [1. 1. 1.]
23     Norma del residuo ||Ax - b||_2: 0.00e+00
24 =====

```

Listing 19: Salida producida por el script de diagnóstico

[▲ Volver al Índice General](#)

## 12 Decálogo de Buenas Prácticas y Conclusiones

1. **Priorice siempre solve sobre inv:** Para resolver  $Ax = b$ , jamás multiplique  $A^{-1}b$ . La función `la.solve(A, b)` es cuatro veces más rápida y no degrada la precisión numérica.
2. **Verifique el número de condición ( $\kappa$ ):** Antes de confiar en los resultados de matrices experimentales o de grandes dimensiones, evalúe `la.cond(A)`. Si  $\kappa(A) > 10^{14}$ , emplee regularización de Tikhonov o descomposiciones basadas en SVD de rango truncado.
3. **Utilice eig para matrices simétricas o hermíticas:** Siempre que  $A = A^T$ , `la.eig` asegura autovalores puramente reales y computa una base ortogonal mediante rutinas LAPACK dedicadas (`dsyevd`).
4. **Prefiera scipy.linalg para operaciones avanzadas:** Brinda soporte directo para descomposiciones LU, matrices en bandas, funciones matriciales como `expm` y acceso a subrutinas optimizadas BLAS/LAPACK.
5. **Nunca compare igualdad en punto flotante con ==:** Debido a los límites de representación del estándar IEEE 754, emplee `np.allclose(A, B, rtol=1e-9, atol=1e-12)`.
6. **Aproveche la estructura matricial:** Si sabe que la matriz es simétrica definida positiva, aplique `cholesky`; si es ortogonal, aproveche que  $Q^{-1} = Q^T$ ; si es triangular, emplee `scipy.linalg.solve_triangular`.
7. **Evite la clase en desuso np.matrix:** Utilice siempre arrays multidimensionales estándar `np.ndarray` en conjunto con el operador `@`.

[▲ Volver al Índice General](#)

## Referencias y Lecturas Recomendadas

---

- [1] **Golub, G. H., & Van Loan, C. F.** (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press.
- [2] **Trefethen, L. N., & Bau, D.** (1997). *Numerical Linear Algebra*. SIAM: Society for Industrial and Applied Mathematics.
- [3] **Strang, G.** (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
- [4] **Higham, N. J.** (2002). *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.
- [5] **Harris, C. R., Millman, K. J., van der Walt, S. J., et al.** (2020). *Array programming with NumPy*. Nature, 585(7825), 357362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [6] **Virtanen, P., Gommers, R., Oliphant, T. E., et al.** (2020). *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17(3), 261272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [7] **NumPy Documentation.** *Linear algebra routines (`numpy.linalg`)*. <https://numpy.org/doc/stable/reference/routines.linalg.html>.
- [8] **SciPy Documentation.** *Linear algebra (`scipy.linalg`)*. <https://docs.scipy.org/doc/scipy/reference/linalg.html>.

▲ [Volver al Índice General](#)